Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:07 Uhr

ANHANG

K Wichtige Bibliotheksklassen

Die Java-Plattform beinhaltet eine große Anzahl von Bibliotheken, die ein weites Spektrum an Programmieraufgaben unterstützen.

In diesem Anhang geben wir eine kurze Übersicht einiger Klassen und Interfaces aus den wichtigsten Paketen des Java-API. Ein kompetenter Java-Programmierer sollte mit den meisten von ihnen vertraut sein. Dieser Anhang gibt lediglich einen Überblick und sollte in Verbindung mit der vollständigen API-Dokumentation gelesen werden.

K.1 Das Paket java.lang

Die Klassen und Interfaces im Paket `java.lang` sind grundlegend für die Sprache Java. Deshalb ist dieses Paket implizit und automatisch in jeder Klassendefinition importiert.

Tabelle K.1
Paket `java.lang` – Kurzdarstellung der wichtigsten Klassen.

Klasse / Interface	Beschreibung
Interface `Comparable`	Die Implementierung dieses Interface ermöglicht den Vergleich und die Sortierung von Objekten der implementierenden Klasse. Statische Hilfsmethoden wie `Arrays.sort` und `Collections.sort` können für Objekte von Typen, die `Comparable` implementieren, zum effizienten Sortieren genutzt werden.
Interface `Runnable`	Ein Interface, das keine Parameter hat und kein Ergebnis zurückgibt, es wird als Typ für Lambda-Ausdrücke mit diesen Eigenschaften verwendet.
Klasse `Math`	Die Klasse `Math` enthält ausschließlich statische Datenfelder und Methoden. Die Werte der mathematischen Konstanten `e` und sind hier definiert, zusammen mit den trigonometrischen und anderen Funktionen, wie beispielsweise `abs` (Betrag), `min`, `max` und `sqrt` (Quadratwurzel).
Klasse `Object`	Alle Klassen haben `Object` als die Wurzelklasse ihrer eigenen Klassenhierarchie. Von dieser Klasse erben alle Objekte Standardimplementierungen von so wichtigen Methoden wie `equals` und `toString`. Weitere wichtige Methoden in dieser Klasse sind `clone` und `hashCode`.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:07 Uhr

Klasse / Interface	Beschreibung
Klasse `String`	Strings spielen in vielen Anwendungen eine wichtige Rolle, sie erhalten deshalb eine besondere Behandlung in Java. Die zentralen Methoden der Klasse `String` sind `charAt`, `equals`, `indexOf`, `length`, `split` und `substring`. Strings sind unveränderliche Objekte, sodass Methoden wie `trim`, die wie verändernde Methoden aussehen, tatsächlich ein neues `String`-Objekt zurückliefern, das das Ergebnis der Operation darstellt.
Klasse `StringBuilder`	Die Klasse `StringBuilder` ist eine effiziente Alternative zu `String`, wenn eine Zeichenkette aus mehreren Komponenten zusammengesetzt werden soll, etwa über Konkatenation. Die zentralen Methoden sind `append`, `insert` und `toString`.

K.2 Das Paket java.util

Das Paket `java.util` ist eine relativ zusammenhangslose Sammlung von nützlichen Klassen und Interfaces.

Tabelle K.2 Paket `java.util` – Kurzdarstellung der wichtigsten Klassen und Interfaces.

Klasse / Interface	Beschreibung
Interface `Collection`	Dieses Interface definiert den Basissatz an Methoden der meisten Sammlungsklassen im Paket `java.util`, wie beispielsweise `ArrayList`, `HashSet` und `LinkedList`. Es definiert die Signaturen für die Methoden `add`, `clear`, `iterator`, `remove` und `size`.
Interface `Iterator`	`Iterator` definiert eine einfache und konsistente Schnittstelle für das Iterieren über den Inhalt einer Sammlung. Die drei wichtigsten Methoden dieses Interface sind `hasNext`, `next`, und `remove`.
Interface `List`	`List` erweitert das Interface `Collection` und definiert Methoden zur expliziten Manipulation der Ordnung in einer Sammlung. Viele Methoden akzeptieren deshalb einen Index als Parameter, wie beispielsweise `add`, `get`, `remove` und `set`. Klassen wie `ArrayList` und `LinkedList` implementieren `List`.
Interface `Map`	Das Interface `Map` bietet eine Alternative zu den listenbasierten Sammlungen, da es die Idee unterstützt, jedes Objekt in einer Sammlung mit einem Schlüsselwert zu verbinden. Objekte werden über ihre `put` und `get`-Methoden hinzugefügt bzw. herausgeholt. Beachten Sie, dass eine `Map` keinen `Iterator` zurückgibt, sondern dass ihre Methode `keySet` einen Satz Schlüssel und ihre Methode `values` eine Sammlung von Objekten in der Map zurückliefert.
Interface `Set`	`Set` erweitert das Interface `Collection` um den Anspruch, dass es Sammlungen ohne Duplikate geben kann. Es muss betont werden, dass `Set` keinerlei Garantie für diese Restriktion liefern kann.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:07 Uhr

Klasse / Interface	Beschreibung
	Das bedeutet, dass **Set** lediglich als Marker-Interface angeboten wird, mit dem die Implementierer von Sammlungsklassen signalisieren können, dass ihre Klassen diese spezielle Einschränkung erfüllen.
Klasse **ArrayList**	**ArrayList** ist eine Implementierung des Interface **List**, die mithilfe von Arrays über **int**-Indizes einen effizienten direkten Zugriff auf die gespeicherten Elemente bietet. Wenn Objekte an anderen Positionen als am Ende eingefügt oder entfernt werden, dann müssen alle nachfolgenden Elemente verschoben werden, um Platz zu schaffen oder die entstehende Lücke zu schließen. Zentrale Methoden sind **add**, **get**, **iterator**, **remove** und **size**.
Klasse **Collections**	**Collections** ist eine Sammelstelle für statische Methoden, mit denen Sammlungen manipuliert werden können. Zentrale Methoden sind **binarySearch**, **fill** und **sort**.
Klasse **HashMap**	**HashMap** ist eine Implementierung des Interface **Map**. Zentrale Methoden sind **get**, **put**, **remove** und **size**. Eine Iteration über eine **HashMap** erfolgt üblicherweise in zwei Schritten: Frage zuerst die Schlüsselmenge über die Methode **keySet** ab, anschließend iteriere über die Schlüssel.
Klasse **HashSet**	**HashSet** ist eine Hashtabellen-basierte Implementierung des Interface **Set**. Zentrale Methoden sind **add**, **remove** und **size**.
Klasse **LinkedList**	**LinkedList** ist eine Implementierung des Interface **List**, die intern eine verkettete Struktur zum Speichern der Elemente verwendet. Direkter Zugriff auf die Enden der Liste ist effizient, aber der Zugriff auf beliebige Elemente über einen Index ist weniger effizient als bei einer **ArrayList**. Andererseits erfordert das Einfügen oder Entfernen innerhalb der Liste nicht das Verschieben von Elementen. Zentrale Methoden sind **add**, **getFirst**, **getLast**, **iterator**, **removeFirst**, **removeIf**, **removeLast**, **size** und **stream**.
Klasse **Random**	Die Klasse **Random** unterstützt die Erzeugung von Pseudozufallswerten, typischerweise von Zufallszahlen. Die Sequenz der erzeugten Zahlen wird durch einen Initialisierungswert (*seed*) bestimmt, der an den Konstruktor übergeben oder über die Methode **setSeed** gesetzt werden kann. Zwei **Random**-Objekte, die vom gleichen Seed-Wert starten, liefern dieselbe Sequenz von Werten bei identischen Aufrufen. Zentrale Methoden sind **nextBoolean**, **nextDouble**, **nextInt** und **setSeed**.
Klasse **Scanner**	Die Klasse **Scanner** bietet Unterstützung beim Einlesen und Zerlegen von Eingaben. Sie wird häufig verwendet, um Eingaben von der Tastatur einzulesen. Zentrale Methoden sind **next** und **hasNext**.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:07 Uhr

K.3 Die Pakete java.io und java.nio.file

Die Pakete **`java.io`** und **`java.nio.file`** enthalten Klassen und Interfaces zur Unterstützung von Eingabe und Ausgabe und von Zugriff auf das Dateisystem. Viele der Ein-/Ausgabeklassen in **`java.io`** sind danach klassifiziert, dass sie entweder auf Datenströmen basieren also auf binären Daten arbeiten oder **`Reader`**- bzw. **`Writer`**-Klassen sind, die auf (menschenlesbaren) Zeichen arbeiten. Im Paket **`java.nio`** finden Sie verschiedene Klassen, die den bequemen Zugriff auf das Dateisystem unterstützen.

Tabelle K.3
Paket **`java.io`** – Kurzdarstellung der wichtigsten Klassen und Interfaces.

Klasse / Interface	Beschreibung
Interface **`Serializable`**	Das Interface **`Serializable`** ist ein leeres Interface, das keinen Code in einer implementierenden Klasse erfordert. Klassen implementieren dieses Interface, um bei einer Serialisierung berücksichtigt werden zu können. Objekte dieser Klassen können als Ganzes aus Eingabequellen und in Ausgabekanäle geschrieben werden. Dies macht das Speichern und Wiederherstellen von persistenten Daten zu einem relativ einfachen Prozess in Java. Siehe auch die Klassen **`ObjectInputStream`** und **`ObjectOutputStream`** für weitere Informationen.
Klasse **`BufferedReader`**	**`BufferedReader`** ist eine Klasse, die gepufferten, zeichenbasierten Zugriff auf eine Eingabequelle bietet. Gepufferte Eingabe ist häufig effizienter als ungepufferte, insbesondere, wenn die Eingabequelle im externen Dateisystem liegt. Weil die Eingabe gepuffert wird, kann diese Klasse die Methode **`readLine`** zum Einlesen einer ganzen Zeile anbieten, die in den meisten anderen Eingabeklassen nicht vorhanden ist. Zentrale Methoden sind **`close`**, **`read`** und **`readLine`**.
Klasse **`BufferedWriter`**	**`BufferedWriter`** ist eine Klasse, die gepufferte, zeichenbasierte Ausgabe anbietet. Gepufferte Ausgabe ist meist effizienter als ungepufferte, insbesondere, wenn das Ausgabeziel im externen Dateisystem liegt. Zentrale Methoden sind **`close`**, **`flush`** und **`write`**.
Klasse **`File`**	Die Klasse **`File`** bietet eine Objektabstraktion für Dateien und Ordner (Verzeichnisse) in einem externen Dateisystem. Sie bietet Methoden zum Prüfen, ob eine Datei lesbar und/oder beschreibbar ist und ob es sich um eine Datei oder ein Verzeichnis handelt. Ein **`File`**-Objekt kann auch für eine nicht vorhandene Datei erzeugt werden, häufig ist dies der erste Schritt für die Erzeugung einer echten Datei in einem Dateisystem. Zentrale Methoden sind **`canRead`**, **`canWrite`**, **`createNewFile`**, **`createTempFile`**, **`getName`**, **`getParent`**, **`getPath`**, **`isDirectory`**, **`isFile`** und **`listFiles`**.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:07 Uhr

Klasse / Interface	Beschreibung
Klasse `FileReader`	Die Klasse `FileReader` wird benutzt, um eine externe Datei zum zeichenweisen Einlesen ihres Inhaltes vorzubereiten. Ein `FileReader`-Objekt wird meist an den Konstruktor einer anderen Klasse übergeben (etwa an `BufferedReader`) und selten direkt benutzt. Zentrale Methoden sind `close` und `read`.
Klasse `FileWriter`	Die Klasse `FileWriter` wird benutzt, um eine externe Datei zum Schreiben von zeichenbasierten Daten vorzubereiten. Über Konstruktorparameter kann festgelegt werden, ob die Daten an den existierenden Inhalt angehängt werden oder ob der Inhalt überschrieben wird. Ein `FileWriter`-Objekt wird meist an den Konstruktor einer anderen Klasse übergeben (etwa an `BufferedWriter`) und selten direkt benutzt. Zentrale Methoden sind `close`, `flush` und `write`.
Klasse `IOException`	`IOException` ist eine Klasse, die geprüfte Exceptions definiert und die Wurzel für die Hierarchie der meisten Exceptionklassen für Ein-/Ausgabe darstellt.

Klasse / Interface	Beschreibung
Interface `Path`	Das Interface `Path` bietet wichtige Methoden für den Zugriff auf Informationen über eine Datei im Dateisystem. `Path` ist genau genommen ein Ersatz für die ältere Klasse `File` im Paket `java.io`.
Klasse `Paths`	Die Klasse `Paths` wird benutzt, um mit `get`-Methoden konkrete Instanzen des `Path`-Interface zurückzuliefern.
Klasse `Files`	Die Klasse `Files` enthält statische Methoden, um Attribute von Dateien und Verzeichnissen (Ordnern) abzufragen und das Dateisystem zu manipulieren, z.B. Verzeichnisse erstellen und Dateizugriffsrechte ändern. Hier finden Sie auch Methoden zum Öffnen von Dateien, wie `newBufferedReader`.

Tabelle K.4
Paket `java.no.file` – Kurzdarstellung der wichtigsten Klassen und Interfaces.

K.4 Das Paket java.util.function

Das Paket `java.util.function` enthält nur Interfaces. Es liefert die häufig vorkommenden Typen, die mit Lambda-Ausdrücken und Methodenreferenzen in Zusammenhang stehen. Darunter sind `Consumer`-, `Supplier`- und `Predicate`-Interfaces sowie die Interfaces für binäre Operatoren. Alle Interfaces sind wichtig und wir beschreiben hier nur eine kleine Auswahl.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:07 Uhr

Tabelle K.5
Paket `java.util.function` – Kurzdarstellung einiger typischer Interfaces.

Klasse / Interface	Beschreibung
Interface `BiFunction`	Das Interface `BiFunction` hat zwei Parameter und liefert einen Ergebniswert zurück. Die Typen von Paramter und Ergebnis können alle unterschiedlich sein, dies ist also eine Generalisierung des `BinaryOperator`-Interface.
Interface `BinaryOperator`	Das Interface `BinaryOperator` hat zwei Parameter seines parametrisierten Typs und liefert ein Ergebnis vom selben Typ. Wird häufig in *reduce*-Operationen eingesetzt.
Interface `Consumer`	Das Interface `Consumer` hat einen Parameter seines parametrisierten Typs und `void` als Ergebnis.
Interface `Predicate`	Das Interface `Predicate` hat einen Parameter seines parametrisierten Typs und liefert ein Ergebnis vom Typ `boolean`. Wird häufig in *filter*-Operationen eingesetzt.
Interface `Supplier`	Das Interface `Supplier` hat keine Parameter und liefert einen Ergebniswert seines parametrisierten Typs zurück.

K.5 Das Paket java.net

Das Paket `java.net` enthält Klassen und Interfaces zur Unterstützung der Netzwerkprogrammierung. Die meisten von ihnen liegen außerhalb des Rahmens dieses Buches.

Tabelle K.6
Paket `java.net` – Kurzdarstellung der wichtigsten Klassen.

Klasse / Interface	Beschreibung
Klasse `URL`	Die Klasse `URL` repräsentiert sogenannte Uniform Resource Locator, d.h., sie bietet die Möglichkeit, die Position einer Ressource im Internet zu beschreiben. Tatsächlich kann sie auch benutzt werden, um eine Ressource in einem lokalen Dateisystem zu beschreiben. Wir haben diese Klasse hier aufgenommen, weil Klassen aus den Paketen `java.io` und `javax.swing` häufig `URL`-Objekte benutzen. Zentrale Methoden sind `getContent`, `getFile`, `getHost`, `getPath` und `openStream`.

K.6 Weitere wichtige Pakete

Weitere wichtige Pakete sind

```
java.awt
java.awt.event
javax.swing
javax.swing.event
```

Diese werden ausführlich bei der Erstellung grafischer Benutzungsoberflächen (GUIs) benutzt und enthalten viele nützliche Klassen, mit denen sich ein GUI-Programmierer vertraut machen sollte.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:07 Uhr